

Lesson 2

Loops, conditional statements and Command line argument



LOOPS/CONDITIONAL STATEMENTS

- Flow Control
- Loops



PYTHON FLOW CONTROL

Decision making structures requires the programmer to specify one or more conditions to be evaluated or tested by the program, along with a statement or statements to be executed if the condition is determined to be true, and optionally, other statements to be executed if the condition is determined to be false.

Python provides following types of decision-making statements:

Statement	Description
if statement	To specify a block of code to be executed, if a specified condition is only true
if...else statement	To specify a block of code to be executed, if the same condition is either false or false
if...elif...else statements	To specify a block of code to be executed, if the same condition is either false or false or an extra condition
Nested if statements statement	You can use one if statement inside another if statement(s)

PYTHON FLOW CONTROL– IF STATEMENT

If the number is positive, we print an appropriate message

```
num = 30
if num > 0:
    print(num, "is a positive number.")
print("Outside print 1.")
```

```
num = -10
if num > 0:
    print(num, "Enter the scene.")
print("Outside print 12")
```

An if statement allows you to take different paths of logic, depending on a given condition. When the condition evaluates to a Boolean true, a block of code for that true condition will execute. You have the option of a single if statement, multiple else if statements, and an optional else statement.

Output

```
30 is positive number
Outside print 1
Outside print 2
```

PYTHON FLOW CONTROL– IF-ELSE STATEMENT

Program checks if the number is
positive or negative
And displays an appropriate
message

```
num = 10
```

```
if num >= 0:  
    print("Positive or Zero")  
else:  
    print("Negative number")
```

Try using -60 and 0

PYTHON FLOW CONTROL– IF-ELIF-ELSE STATEMENT

'''In this program,
we check if the number is positive or
negative or zero and
display an appropriate message'''

```
num = 5.87
```

```
# Try these two variations as well:
```

```
if num > 0:  
    print("Positive number")  
elif num == 0:  
    print("Zero")  
else:  
    print("Negative number")
```

Try using -60 and 0

PYTHON FLOW CONTROL– NESTED IF STATEMENT

'''In this program, we input a number
check if the number is positive or
negative or zero and display
an appropriate message
This time we use nested if statement'''

```
num = float(input("Enter a number: "))  
if num >= 0:  
    if num == 0:  
        print("Zero")  
    else:  
        print("Positive number")  
else:  
    print("Negative number")
```

Indentation is the only way
to figure out the level of
nesting. They can get
confusing, so they must be
avoided unless necessary.

Try testing with 100, -60 and
0



COMPLETE THE FOLLOWING QUESTION (10MINS)

- Write a program to accept the cost price of a car and display the tax to be paid according to the following criteria :

Assumptions:

- You can assume that the input will be a number.

Cost price (in CAD)	Tax
> 200000	15%
> 50000 and <= 200000	10%
<= 50000	5%

LOOPS

A loop statement allows us to execute a statement or a group of statements.

Loop Type	Description
WHILE Loop	This repeats a statement or a group of statements while a given condition is true. It tests the condition before executing the loop body
FOR Loop	The for loop in Python is used to iterate over a sequence (list, tuple, string) or other iterable objects
Nested Loop	One or more loops inside another loop.

Syntax

```
Counter=1  
while(condition):  
    statement(s);  
    counter +=1
```

Syntax

```
for val in sequence:  
    statement(s);
```

Syntax

```
Counter=1  
while(condition):  
    statement(s);  
    for val in sequence:  
        statement(s);  
    counter +=1
```

LOOPS – WHILE

FOR LOOP

```
for i in range(1, 6):  
    print(i)
```

```
print(range(6))
```

WHILE LOOP

```
i = 1
```

```
while i <= 5:  
    print(i)  
    i = i+1 # update counter
```

Output

```
1  
2  
3  
4  
5
```

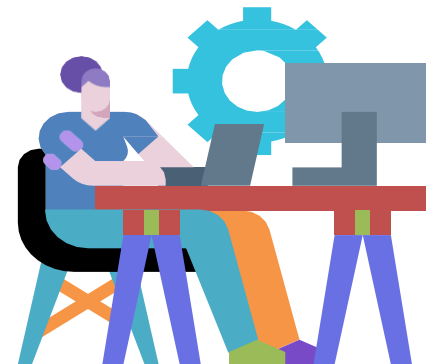
COMPLETE THE FOLLOWING QUESTION

1. Create an array that contains these values 6, 5, 3, 8, 4, 2, 5, 4, 11
2. Add and store the sum of the all the numbers
3. Format your output to the below
“The sum of all the numbers in the list is 48”

Command line argument

COMMAND LINE ARGUMENT

- Break and continue
- Command line Argument



PYTHON BREAK AND CONTINUE

Break

A break statement can also be used to jump out of a loop.

Use of break statement inside the loop

```
for val in "Hello":  
    if val == "e":  
        break  
    print(val)  
  
print("The end")
```

PYTHON BREAK AND CONTINUE

Continue

The continue statement breaks one iteration (in the loop), if a specified condition occurs, and continues with the next iteration in the loop.

```
# Use of break statement inside the loop
```

```
for val in "Hello":  
    if val == "e":  
        continue  
    print(val)  
  
print("The end")
```

COMPLETE THE FOLLOWING QUESTION

Write a program that filters even and odd numbers from the below list.

Given $x = [10, 23, 24, 35, 65, 78, 100]$

Tips: All odd numbers should be stored and printed in one list and all even numbers should be stored and printed in another list.

Expected output

Even Nos: $[10, 24, 78, 100]$

Odd Nos: $[23, 35, 65]$

COMMAND LINE ARGUMENTS

The arguments are the parameters given after the name of the program in the command line shell of the operating system. We use the following below to deal with these types of arguments.

sys module

getopt module

argparse module

COMMAND LINE ARGUMENTS (SYS.ARGV)

The sys module provides functions and variables used to manipulate different parts of the Python runtime environment. This module provides access to some variables used or maintained by the interpreter and to functions that interact strongly with the interpreter.

One such variable is sys.argv which is a simple list structure. It's main purpose are:

- It is a list of command line arguments.
- len(sys.argv) provides the number of command line arguments.
- sys.argv[0] is the name of the current Python script.

NB: To run command-line argument, go to either PowerShell or Command Prompt, Navigate to the location where your file exist, Run the command: **Python {nameofPythonFile} {Arguments}**

COMMAND LINE ARGUMENTS (SYS.ARGV)

```
import sys

# total arguments
total_no_arguments = len(sys.argv)
print("Total arguments passed:",
      total_no_arguments)

# Arguments passed
print("\nName of Python script:",
      sys.argv[0])

print("\nArguments passed:", end=" ")
for i in range(1, total_no_arguments):
    print(sys.argv[i], end=" ")

# Addition of numbers
Sum = 0
# Using argparse module
for i in range(1, total_no_arguments):
    Sum += int(sys.argv[i])

print("\n\nResult:", Sum)
```

Create a python file with name
“commandlineDemonstration.py”

Python *commandlineDemonstration.py* 4 4 4

COMMAND LINE ARGUMENTS (GETOPT MODULE)

The second approach in python to implement command-line arguments is by using the getopt module.

getopt module provides two functions and an exception to facilitate command-line argument parsing.

Standard libraries are imported. **Sys** module for getting command-line arguments and **getopt** module for parsing of command-line arguments.

Syntax: getopt.getopt(args, options, [long_options])

Parameters:

args: List of arguments to be passed.

options: String of option letters that the script want to recognize. Options that require an argument should be followed by a colon (:).

long_options: List of string with the name of long options. Options that require arguments should be followed by an equal sign (=).

Return Type: Returns value consisting of two elements: the first is a list of (option, value) pairs. The second is the list of program arguments left after the option list was stripped.

COMMAND LINE ARGUMENTS (GETOPT MODULE)

```
# importing standard libraries
import getopt
import sys
# extracting command line arguments excluding
file name
argv = sys.argv[1:]
# parsing the argument list
try:
    opts, args = getopt.getopt(argv, 'ho:v')
    print(opts)
    print(args)
except getopt.GetoptError:
    print('Something went wrong!')
    sys.exit(2)
```

Create a python file with name
“commandlineDemonstration2.py”

Python *commandlineDemonstration2.py -o
Colors -h Blue Red Yellow Green*

COMMAND LINE ARGUMENTS (ARGPARSE MODULE)

Python **argparse** module is the most preferred way to parse command-line arguments as it provides a lot of options such as positional arguments, the default value for arguments, help message, etc.

The first **argparse** module is imported. Then an object of `argparse.ArgumentParser()` is created. When a program with `argparse` executed with any input, nothing happens.

Argparse supports two types of arguments:

1. Positional arguments
2. Optional arguments

COMMAND LINE ARGUMENTS (SYS.ARGV)

```
# importing argparse module
import argparse
# creating parser object
parser =
argparse.ArgumentParser(description="Mes
sage description")
# adding positional arguments
parser.add_argument("first_number",help=
'Enter number 1 as your first argument')
parser.add_argument("second_number",h
elp='Enter number 2 as your first argument')
args = parser.parse_args()
# type cast the arguments as default is
string type
first_number = int(args.first_number)
second_number =
int(args.second_number)
print("The sum of the two number
is",first_number+second_number)
```

Create a python file with name
“commandlineDemonstration3.py”

Demonstration on command Line :

Python *commandlineDemonstration3.py* -h
(for help)

Python *commandlineDemonstration3.py* 5 10

COMPLETE THE FOLLOWING QUESTION

Write a program that filters even numbers from the below list of command line argument.

Command Line argument: 10 23 24 35 65 78 100

COMPLETE THE FOLLOWING QUESTION

Write a program that takes yyyy mm dd as command line argument and prints the next day of the giving date. The program should return invalid date if the user pass in an invalid date.

Tips:

1. Ensure you check for leap to determine if February is 28 or 29
2. Check the month as January, March, May, July, August, October and December has 31 days and April, June, September, November has 30 days

Test your code with the following values

Valid Test:

1988 02 29

2022 12 31

2021 04 30

Invalid Test:

2019 05 32

1989 02 29

2019 21 03